

Plan: Splitting `main.js` into Modules

1. Goal and Constraints

`editor-src/src/main.js` is 1305 lines. This plan splits it into focused source modules that a central `main.js` imports and assembles. Vite bundles them back into one file for `editor.html`, so this is a source-organization change with no runtime cost and no change to the Swift bridge.

Two constraints shape every decision below:

- There is exactly one `EditorView`, held in the module-level `view` constant, and one extension array. `main.js` stays the assembly point that owns `view`, builds the extension array, and exposes `window.editorBridge`. Modules produce the pieces that feed into it; they never construct or hold the view.
- ES module imports are read-only live bindings. A module cannot reassign a value it imported from another module. Any mutable value shared across a module boundary must therefore live behind an object whose property is mutated, not a bare exported `let`. This is what `state.js` is for.

2. The `state.js` Foundation Module

`state.js` is the dependency-free module that every other module may import without risking a cycle. It holds two things.

2.1. The `post` helper

`post(msg)` is the JS-to-Swift sender. It is currently used by the update listener and scroll tracking (both staying in `main.js`) and by `openLink` (moving to `links.js`). Because more than one module needs it, it lives here rather than in `main.js`, so `links.js` can import `post` without importing `main.js` and creating a `main → links → main` cycle.

2.2. The shared mutable state object

A single object holds the cross-cutting mutable flags, replacing today's module-level lets:

- `state.suppressDocChanged`
- `state.fontSize` (today `currentFontSize`)
- `state.fontFamily` (today `currentFontFamily`)
- `state.autoScrollMode`

A note on scope. With the module boundaries proposed here, `main.js` ends up being the only reader and writer of all four flags, because the `config` helpers, the `bridge`, and `doCenteredScroll` all stay in `main.js`. So strictly, these could remain plain `lets` inside `main.js` today. They are centralized in `state.js` anyway for one documented home and a single source of truth, and so that a later extraction (for example moving the `config` helpers out) does not require reworking how the flags are shared. This is the one place the plan chooses organization over the minimum necessary change; call it out at review time if you would rather keep them local in `main.js` for now.

3. Proposed Module Layout

Each row is a new file under `editor-src/src/`. “Library imports” lists what it pulls from `CodeMirror/Lezer`; “Local imports” lists what it pulls from our own modules; “Exports” is what `main.js` (or a sibling) consumes.

Module	Contents	Library imports	Local imports	Exports
<code>state.js</code>	<code>post</code> , <code>state</code> object	none	none	<code>post</code> , <code>state</code>
<code>parser-extensions.js</code>	<code>footnoteImageFix</code> , <code>plainBracketFix</code> , <code>footnoteDefFix</code> , <code>headingOnlyFold</code>	<code>parser</code> as <code>baseMarkdownParser</code> from <code>@lezer/markdown</code> , <code>foldNodeProp</code>	none	the four e
<code>theme.js</code>	<code>editorBaseTheme</code> , <code>buildFontTheme</code> , <code>fontFamilyCSS</code>	<code>EditorView</code>	none	<code>editorBas</code> <code>buildFont</code>
<code>highlight.js</code>	<code>highlightStyle</code> , <code>conspicuousMark</code> , <code>conspicuousMarkStyle</code>	<code>HighlightStyle</code> , <code>tags</code> , <code>Tag</code> , <code>styleTags</code>	none	<code>highlight</code> <code>conspicu</code>
<code>decorations.js</code>	<code>headingLineDecorations</code> , <code>inlineMarkDecorations</code> , <code>linkDecorations</code> , <code>cmdKeyTracking</code> , plus	<code>ViewPlugin</code> , <code>Decoration</code> , <code>syntaxTree</code>	none	the four V

Module	Contents	Library imports	Local imports	Exports
gutters.js	their build* helpers and fnRefRe wordMilestones, wordCountGutter, MilestoneMarker, computeWordMilestones, wordRe	gutter, GutterMarker, StateField	none	wordMiles wordCount
footnotes.js	collectFootnoteDefs, lookupFootnoteDef, footnoteTipAt, footnoteHover, fnDefRe, fnRefRe	hoverTooltip	none	footnoteH collectFo fnRefRe (I arrangeFo
links.js	slugify, headingPosForSlug, goToHeading, openLink	EditorView	post from state.js	goToHead:
search- panel.js	createSearchPanel	findNext, findPrevious, replaceNext, replaceAll, getSearchQuery, setSearchQuery, SearchQuery from @codemirror/search EditorView, EditorState, Transaction, Compartment, keymap, gutters, drawSelection; markdown, GFM; syntaxHighlighting, foldGutter, foldKeymap; history, defaultKeymap, historyKeymap; search, openSearchPanel, closeSearchPanel, searchPanelOpen, searchKeymap	none	createSea
main.js	imports + view assembly + bottomPadObserver + scroll tracking + doCenteredScroll + config helpers + window.editorBridge		everything from the modules above	window.ec (global sid

4. Notes on Specific Boundaries

4.1. What stays in `main.js` and why

`doCenteredScroll` reads the module-level `view` and `state.autoScrollMode`, and the update listener calls it. The config helpers (`buildCompartmentEffects`, `applyConfigToDOM`, `rgbToRgba`) drive the compartment and DOM off `view` and the font mirrors. The bridge methods (`load`, `updateConfig`, `toggleSearch`, `closeSearch`, `scrollToHeading`, `arrangeFootnotes`, `getContent`) all operate on `view`. All of these are tightly bound to the singleton, so they remain in `main.js` rather than taking `view` as a parameter just to live elsewhere.

4.2. `fnRefRe` appears in two places

The footnote-reference regex is used by `inlineMarkDecorations` (in `decorations.js`) and by `arrangeFootnotes` (in `main.js`, via `footnotes.js`). Define it once in `footnotes.js` and export it; `decorations.js` imports it from there. This avoids two copies drifting apart. Confirm during the move that both call sites reset `lastIndex` as needed, since it is a `/g` regex (today `arrangeFootnotes` does `fnRefRe.lastIndex = 0`).

4.3. `arrangeFootnotes` splits across two modules

The command is a bridge method, so it stays in `main.js`. Its parsing dependencies (`collectFootnoteDefs`, `fnRefRe`) come from `footnotes.js`. The string-rewriting body stays inline in the bridge method; only the shared parsing utilities are imported.

4.4. `goToHeading` has two callers

`links.js` uses `goToHeading` internally (from `openLink` on a `#fragment`) and `main.js` calls it from the `scrollToHeading` bridge method. Export it from `links.js`; `main.js` imports both `goToHeading` and `openLink`.

4.5. Search imports divide cleanly

`createSearchPanel` needs the query commands and effects (`findNext`, `findPrevious`, `replaceNext`, `replaceAll`, `getSearchQuery`, `setSearchQuery`, `SearchQuery`), so those imports move to `search-panel.js`. `main.js` keeps the imports it uses for assembly and the bridge: `search`, `searchKeymap`, `openSearchPanel`, `closeSearchPanel`, `searchPanelOpen`.

5. Migration Order

Do this as a sequence of small, individually verifiable moves rather than one large cut. Build (`vite build` or the project's build step) and load the editor after each step; a blank editor means `new EditorView()` threw during construction.

1. Create `state.js` with `post` and the state object. In `main.js`, import `post` and replace the local `post` and the four `lets` with `state.*` references. Verify nothing else changed.
2. Move the leaf modules with no local dependencies, one at a time: `parser-extensions.js`, `theme.js`, `highlight.js`, `gutters.js`. Each is a lift of a contiguous section plus its imports; update `main.js` to import what it now references.
3. Move `footnotes.js` (exports `fnRefRe`), then `decorations.js` (imports `fnRefRe` from it).
4. Move `links.js` (imports `post` from `state.js`).
5. Move `search-panel.js`.
6. Leave `main.js` as the assembly, observers, config helpers, and bridge. Confirm its import list matches the table in section 3.

6. Risks and How They Show Up

- Circular imports. The only realistic cycle is through `post`; putting it in `state.js` prevents it. If a new cycle appears, the symptom is an undefined import at module-eval time, usually a `TypeError` in the console before `editorReady`.
- A thrown error during `new EditorView()` renders the editor blank (the same failure mode the theming doc warns about). Because the move is staged, any blank-on-build points at the step just made.
- Stale `/g` regex state if `fnRefRe` is shared without resetting `lastIndex`. Verify both call sites after step 3.
- Import-list drift in `main.js`: an unused import is harmless but messy; a missing one fails the build. The section 3 table is the checklist.

7. Verification

After the full split, build and exercise: open a blob, type to confirm `documentChanged` still posts and autoscroll behaves; toggle search and run find/replace; hover a footnote reference; `Cmd+click` an external link and an in-document `#anchor`; run Arrange Footnotes; change font size and family

and a color from Swift to confirm the config patch path; open the mini view to confirm the `mini` class and gutter placement. These cover every module boundary introduced here.

8. Out of Scope

Documentation updates to `docs/cm-editor-customs.md`, which currently states all editor JavaScript lives in `main.js`, are deliberately deferred per the current request.